spring form #2

학습목표

- select 태그
- 체크박스
- 라디오 버튼
- lombok
- 로그 메시지 출력

목 차

1. 배경 지식 2
   1) lombok 2
   2) 로그 기록 3
2. form2 프로젝트 5
   1) 프로젝트 생성 5
   2) application.properties 7
   3) Student.java 7
   4) StudentController.java 8
   5) common.css 9
   6) student/edit.html 10
   7) student/list.html 14
   8) 실행 15
   9) 연습문제 16
3. 부록 17
   1) lombok 17
   2) 로그 메시지 출력 18

1. 배경 지식

1) lombok

lombok 은 getter, setter 메소드를 자동으로 생성해 주는 도구이다.
lombok 을 사용하면, getter, setter 메소드를 구현하지 않아도 되니 편하다.
equals 메소드, hashCode 메소드, toString 메소드도 자동으로 생성해 준다.

DTO 클래스에 @Data 어노테이션을 붙이면
  모든 멤버 변수에 대해서
  getter, setter, equals, hashCode, toString 메소드들이 자동 구현된다.

바이트코드 파일에 생성됨 (*.class)

그런데 소스 코드 파일이 아니고, 컴파일되어 생성되는 *.class 파일에,
위 메소드들이 자동 생성된다.

@Data 어노테이션

이 어노테이션을 클래스 앞에 붙이면,
getter, setter, equals, hashCode, toString 메소드가 자동 구현된다.
예:

```
@Data
public class Student {
   int id;
   String name;
}
```

- 모든 멤버 변수에 대해서 getter setter 가 자동으로 구현된다.
- 모든 멤버 변수 값이 같은지 비교하는 equals 메소드가 자동으로 구현된다.
- 모든 멤버 변수 값을 바탕으로 해시 값을 계산하는 hashCode 메소드가 자동으로 구현된다.
- 모든 멤버 변수 값을 문자열로 표현하는 toString 메소드가 자동으로 구현된다.

lombok 설치

개발도구 설치 강의노트 참고

2) 로그 기록

버그를 찾고 해결하는 가장 좋은 수단은 다음과 같다.
  (1) JUnit 단위 테스트 구현
  (2) 로그 메시지 출력

디버깅에 필요한 정보를 출력할 때,
  System.out.println(...) 메소드를 사용하는 것은 바람직하지 않다.
  로그 메시지 출력 기능을 사용해야 한다.

@Slf4j

로그 메시지 출력 기능을 구현하려는 클래스에 이 어노테이션을 붙여주자.

```
import lombok.extern.slf4j.Slf4j;

@Controller
@Slf4j
public class StudentController {
```

즉 위 코드는 아래의 구현과 동일하다.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Controller
public class StudentController {
    static final Logger log = LoggerFactory.getLogger(StudentService.class);
```

즉 @Slf4j 어노테이션이 붙은 클래스에는 log 멤버 변수 객체가 자동으로 생성된다.
이것도 lombok의 기능이다.
이 기능도, 소스코드를 수정하지 않고, 바이트코드 파일만 수정해서 위와 같은 멤버 변수 객체가 생성된다.

로그 메시지 등급

심각한 정도를 기준으로 로그 메시지의 등급이 구분된다.
  trace < debug < info < warn < error
    trace 등급이 가장 낮고, error 등급이 가장 높다.
    주로 사용하는 등급은 debug, info, error 이다.

    error: 에러 발생
    warn: 에러까지는 아니지만, 큰 성능 하락 등 위험한 상황
    info: 서버 시작, 연결 성공 등 중요한 이벤트
    debug: 디버깅을 위해 필요한 여러가지 정보 출력
    trace: 사소한 정보 출력

등급별 로그 메시지 출력 메소드

```
log.error("An ERROR Message");
log.warn("A WARN Message");
log.info("An INFO Message");
log.debug("A DEBUG Message");
log.trace("A TRACE Message");
```

이 메소드들로 출력한 로그 메시지들은
  eclipse 에서 실행할 때 eclipse 콘솔창에 출력되고,
  서버에서 실행할 때는 로그 파일에 기록된다. (약간의 서버 로그 출력 설정 필요)

로그 메시지 출력 설정하기

소스 코드에 구현한 로그 메시지 출력이 언제나 전부 출력되는 것은 아니다.
어느 등급 이상만 출력하고 그 미만은 무시하라고 설정할 수 있다.

application.properties

```
logging.level.root=info
```

위와 같이 설정하면 info 등급 이상의 로그 메시지만 출력되고,
이 미만 등급은 전부 무시된다.
이 설정의 디폴트 값은 info 등급이다.

```
logging.level.root=debug
```

위와 같이 설정하면 debug 등급 이상의 로그 메시지들이 출력된다.

```
logging.level.net.skhu=debug
```

net.skhu 패키지 아래 클래스들은 debug 등급 이상
그외 나머지 클래스들은 디폴트인 info 등급 이상

```
logging.level.net.skhu.mapper=debug
```

net.skhu.mapper 패키지 아래 클래스들은 debug 등급 이상
그외 나머지 클래스들은 디폴트인 info 등급 이상

```
logging.net.skhu.controller.StudentController=debug
```

net.skhu.controller.StudentController 클래스는 debug 등급 이상
그외 나머지 클래스들은 디폴트인 info 등급 이상

디버깅이 끝나면 위 debug 설정만 지우면 된다.
디폴트 설정이 info 이므로, log.deubg(...) 출력은 무시되기 때문이다.

소스코드에서 log.debug(...) 호출을 삭제하지 않으면,
나중에 또 디버깅 해야할 때, 그 부분의 debug 등급 설정만 해주면 되니 편리하다.

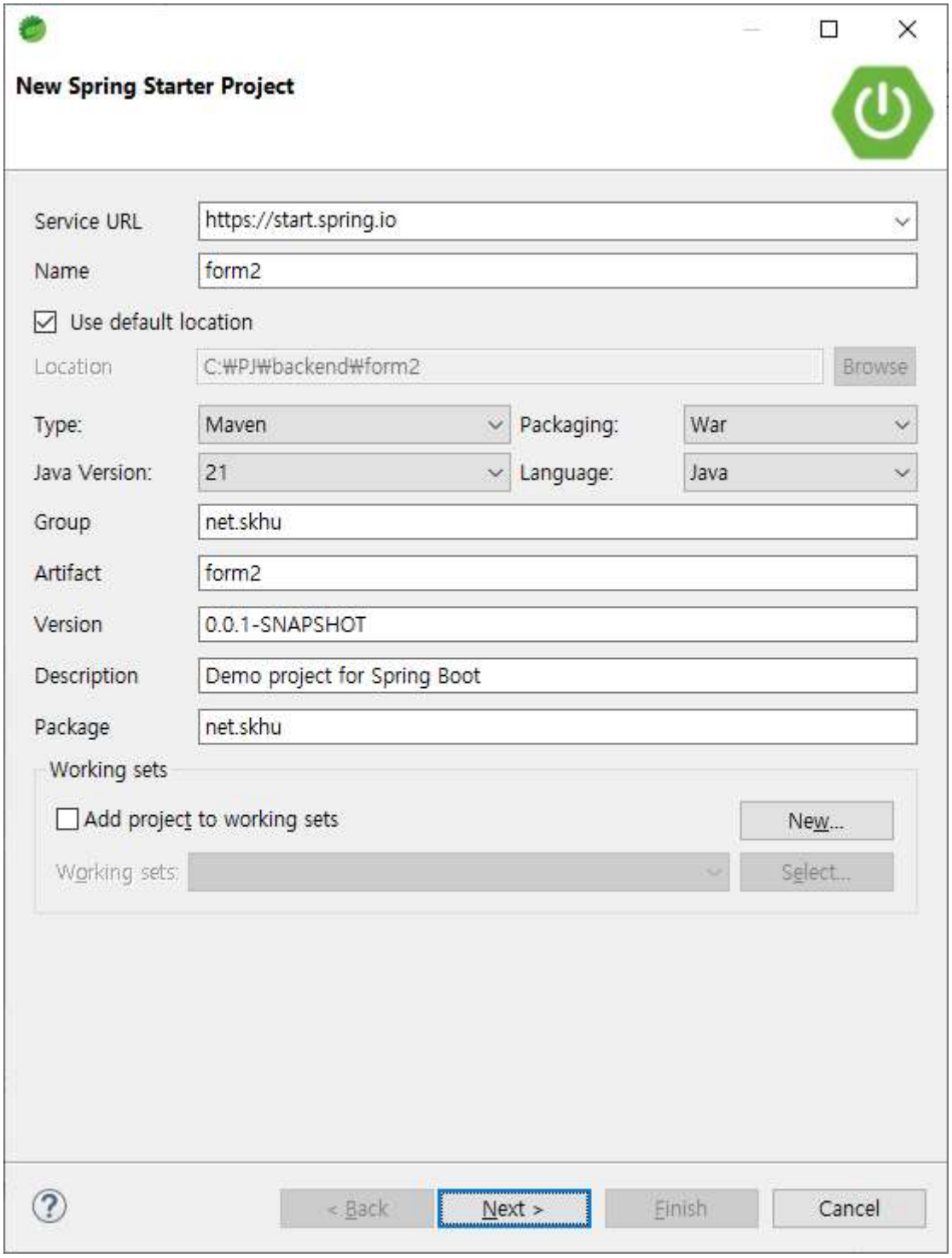
디버깅을 위한 로그 등급 설정을 제거할 때, 완전히 제거하는 대신 주석처리 해도 된다.
application.properties 파일에서 주석은 # 문자로 시작한다.

```
#logging.level.net.skhu=debug
```

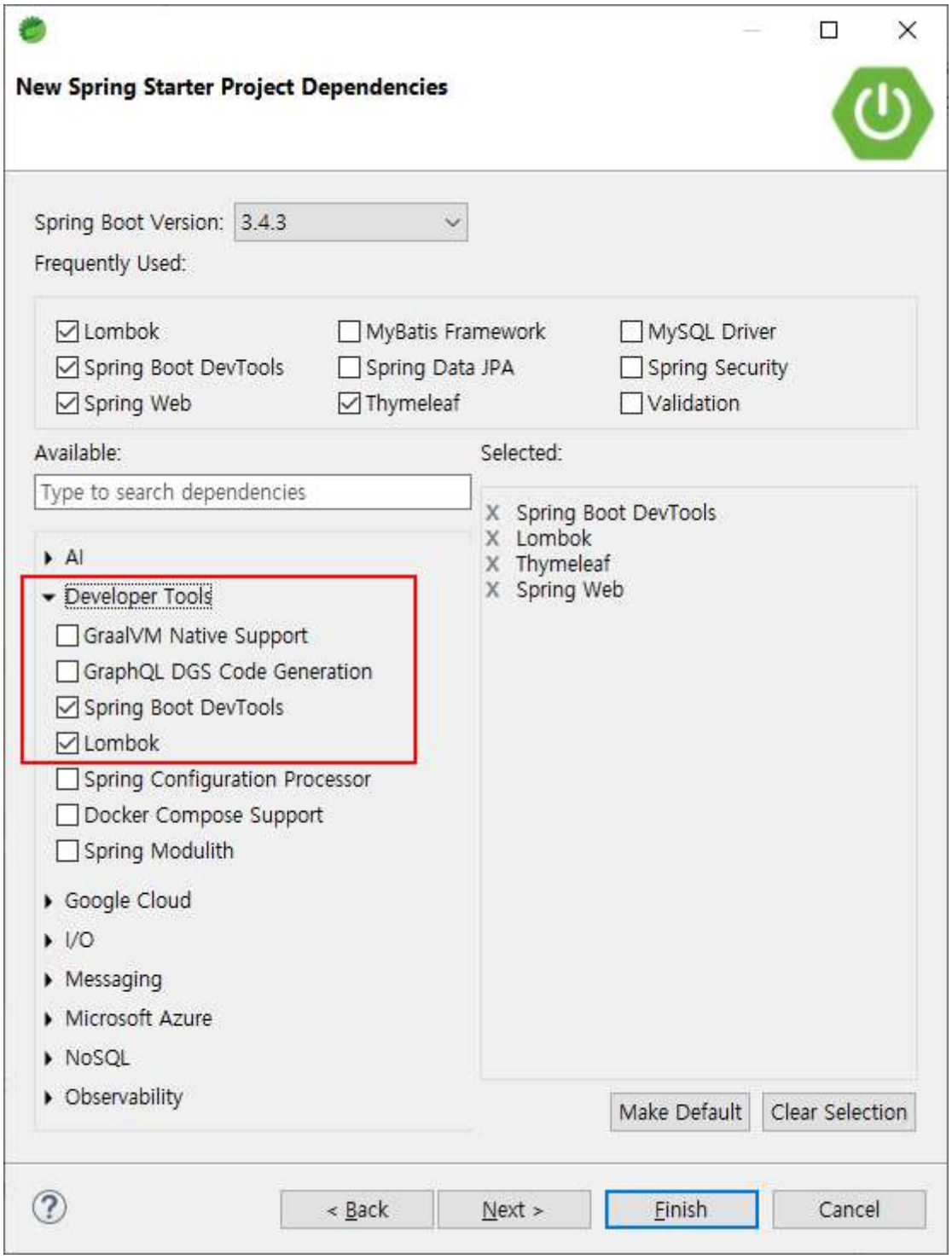
debug 단계 로그 설정이 주석 처리되었다.
따라서 info 등급 이상의 로그 메시지만 출력된다.

2. form2 프로젝트

1) 프로젝트 생성



Name	form2
Type	Maven
Java Version	21
Package	net.skhu



Developer Tools 그룹 아래
  Spring Boot DevTools
  Lombok

Template Enginers 그룹 아래
  Thymeleaf

Web 그룹 아래
  Srping Web

2) application.properties

src/main/resources/application.properties

```
spring.application.name=form2
server.port=8088
logging.level.net.skhu=debug
```

3) Student.java

src/main/java/net/skhu/dto/Student.java

```
1   package net.skhu.dto;
2
3   import lombok.Data;
4
5   @Data
6   public class Student {
7       String studentNo;
8       String name;
9       int departmentId;
10      String gender;
11      boolean absense;
12      int year;
13  }
```

4) StudentController.java

src/main/java/net/skhu/controller/StudentController.java

```java
package net.skhu.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.util.StringUtils;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import lombok.extern.slf4j.Slf4j;
import net.skhu.dto.Student;

@Controller
@RequestMapping("student")
@Slf4j
public class StudentController {

    @GetMapping("edit")
    public String edit(Model model) {
        Student student = new Student();
        student.setName("홍길동");
        log.debug(student.toString());
        model.addAttribute("student", student);
        return "student/edit";
    }

    @PostMapping("edit")
    public String edit(Model model, Student student) {
        try {
            log.debug(student.toString());
            if (StringUtils.hasText(student.getStudentNo()) == false)
                throw new Exception("학번을 입력하세요");
            if (StringUtils.hasText(student.getName()) == false)
                throw new Exception("이름을 입력하세요");

            // student 객체를 DB에 저장하는 구현 생략
            return "redirect:list";
        }
        catch (Exception ex) {
            model.addAttribute("errorMsg", ex.getMessage());
            return "student/edit";
        }
    }

    @GetMapping("list")
    public String list() {
        // 학색 목록을 DB에서 조회해서 뷰에 전달하는 코드 생략
        return "student/list";
    }
}
```

5) common.css

src/main/resources/static/common.css

```
1  div.container { width: 800px; margin: 10px auto; font-size: 11pt; }
2  .shadow { box-shadow: 2px 2px 5px gray; }
3
4  .btn { padding: 0.4em 1em; border: 1px solid gray;
5         border-radius: 0.5em; background: linear-gradient(#fff, #ddd);
6         text-decoration: none; color: black;
7         display: inline-block; }
8  .btn:active {
9        -ms-transform: translateY(2px);
10       -webkit-transform: translateY(2px);
11       transform: translateY(2px);
12       background: #ccc; }
13
14 table.list { border-collapse: collapse; width: 100%; }
15 table.list td { padding: 4px; border: 1px solid gray; }
16 table.list th { padding: 4px; border: 1px solid gray;  background-color: #eee; }
17
18 div.row { margin-bottom: 0.8em; }
19 div.row > label:nth-child(1) { display: inline-block; min-width: 4em; text-align: right; }
20
21 input[type=text], input[type=number] { padding: 0.3em; }
22 select { padding: 0.3em; }
23 h1 { margin: 0.5em 0; }
24 hr { margin: 1em 0; }
25 .error { color: red; padding: 0.2em; margin-left: 1em; }
```

6) student/edit.html

src/main/resources/templates/student/edit.html

```
<!DOCTYPE html>
<html lang="ko" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8">
  <link rel="stylesheet" type="text/css" href="/common.css" />
  <style>
    form { width: 500px; padding: 1em 3em; }
    button { margin-left: 1em; }
  </style>
</head>
<body>
<div class="container">
 <form method="post" th:object="${student}" class="shadow">
    <h1>학생 수정</h1>
    <div class="row">
      <label>학번:</label>
      <input type="text" th:field="*{studentNo}" />
    </div>
    <div class="row">
      <label>이름:</label>
      <input type="text" th:field="*{name}" />
    </div>
    <div class="row">
      <label>학과:</label>
      <select th:field="*{departmentId}">
        <option value="0">학과를 선택하세요</option>
        <option value="1">소프트웨어공학과</option>
        <option value="2">컴퓨터공학과</option>
        <option value="3">정보통신공학과</option>
      </select>
    </div>
    <div class="row">
      <label>성별:</label>
      <label><input type="radio" th:field="*{gender}" value="남자" />남자</label>
      <label><input type="radio" th:field="*{gender}" value="여자" />여자</label>
    </div>
    <div class="row">
      <label>졸업:</label>
      <label><input type="checkbox" th:field="*{absense}" />휴학</label>
    </div>
    <div class="row">
      <label>학년:</label>
      <input type="number" th:field="*{year}" />
    </div>
    <hr />
    <button type="submit" class="btn">저장</button>
    <span class="error" th:text="${ errorMsg }"></span>
  </form>
</div>
</body>
</html>
```

```
<form method="post" th:object="${student}">
     <input type="text" th:field="*{studentNo}" />
     <input type="text" th:field="*{name}" />

     <select th:field="*{departmentId}">
       <option value="0">학과를 선택하세요</option>
       <option value="1">소프트웨어공학과</option>
       <option value="2">컴퓨터공학과</option>
       <option value="3">정보통신공학과</option>
     </select>

     <label><input type="radio" th:field="*{gender}" value="남자" />남자</label>
     <label><input type="radio" th:field="*{gender}" value="여자" />여자</label>

     <label><input type="checkbox" th:field="*{absense}" />휴학</label>
     <input type="number" th:field="*{year}" />

     <button type="submit" class="btn">저장</button>
 </form>
```

입력 태그 요약

```
<form method="post" th:object="${student}">
```
student 이름의 model attribute 객체

텍스트 입력
```
   <input type="text" th:field="*{studentNo}" />
```
student.studentNo 속성
```
   <input type="text" th:field="*{name}" />
```
student.name 속성

선택 목록
```
   <select th:field="*{departmentId}">
     <option value="0">학과를 선택하세요</option>
     <option value="1">소프트웨어공학과</option>
     <option value="2">컴퓨터공학과</option>
     <option value="3">정보통신공학과</option>
   </select>
```
student.departmentId 속성

라디오 버튼
```
   <label><input type="radio" th:field="*{gender}" value="남자" />남자</label>
   <label><input type="radio" th:field="*{gender}" value="여자" />여자</label>
```
student.gender 속성

체크박스
```
   <label><input type="checkbox" th:field="*{absense}" />휴학</label>
```
student.absense 속성

숫자입력
```
   <input type="number" th:field="*{year}" />
```
student.year 속성

submit 버튼
```
   <button type="submit" class="btn">저장</button>
```

라디오 버튼과 체크박스를 <label> 태그로 감싸주면
남자, 여자, 휴학 등 레이블 부분을 클릭해도 체크 상태가 변경되니 편리하다.

html 태그 출력

```
<form method="post" th:object="${student}">
```

```
  <input type="text" th:field="*{studentNo}" />
  <input type="text" th:field="*{name}" />
  <select th:field="*{departmentId}">
    <option value="0">학과를 선택하세요</option>
    <option value="1">소프트웨어공학과</option>
    <option value="2">컴퓨터공학과</option>
    <option value="3">정보통신공학과</option>
  </select>
  <label><input type="radio" th:field="*{gender}" value="남자" />남자</label>
  <label><input type="radio" th:field="*{gender}" value="여자" />여자</label>
  <label><input type="checkbox" th:field="*{absense}" />휴학</label>
  <input type="number" th:field="*{year}" />
```

예를 들어, student model attribute 객체의 속성이 다음과 같을 때

studentNo	201914199
name	홍길동
departmentId	1
gender	남자
absense	true
year	4

위 thymeleaf 소스코드는 다음과 같이 출력된다.

```
<form method="post">
  <input type="text" name="studentNo" value="201914199" />
  <input type="text" name="name" value="홍길동" />
  <select name="departmentId">
    <option value="0">학과를 선택하세요</option>
    <option value="1" selected>소프트웨어공학과</option>
    <option value="2">컴퓨터공학과</option>
    <option value="3">정보통신공학과</option>
  </select>
  <label><input type="radio" name="gender" value="남자" checked />남자</label>
  <label><input type="radio" name="gender" value="여자" />여자</label>
  <label><input type="checkbox" name="absense" value="true" checked />휴학</label>
  <input type="number" name="year" value="4" />
```

예를 들어, student model attribute 객체의 속성이 다음과 같을 때

studentNo	201914199
name	성춘향
departmentId	2
gender	여자
absense	false
year	3

위 thymeleaf 소스코드는 다음과 같이 출력된다.

```
<form method="post">
  <input type="text" name="studentNo" value="201914199" />
  <input type="text" name="name" value="성춘향" />
  <select name="departmentId">
    <option value="0">학과를 선택하세요</option>
    <option value="1">소프트웨어공학과</option>
    <option value="2" selected>컴퓨터공학과</option>
    <option value="3">정보통신공학과</option>
  </select>
  <label><input type="radio" name="gender" value="남자" />남자</label>
  <label><input type="radio" name="gender" value="여자" checked />여자</label>
  <label><input type="checkbox" name="absense" value="true" />휴학</label>
  <input type="number" name="year" value="3" />
```

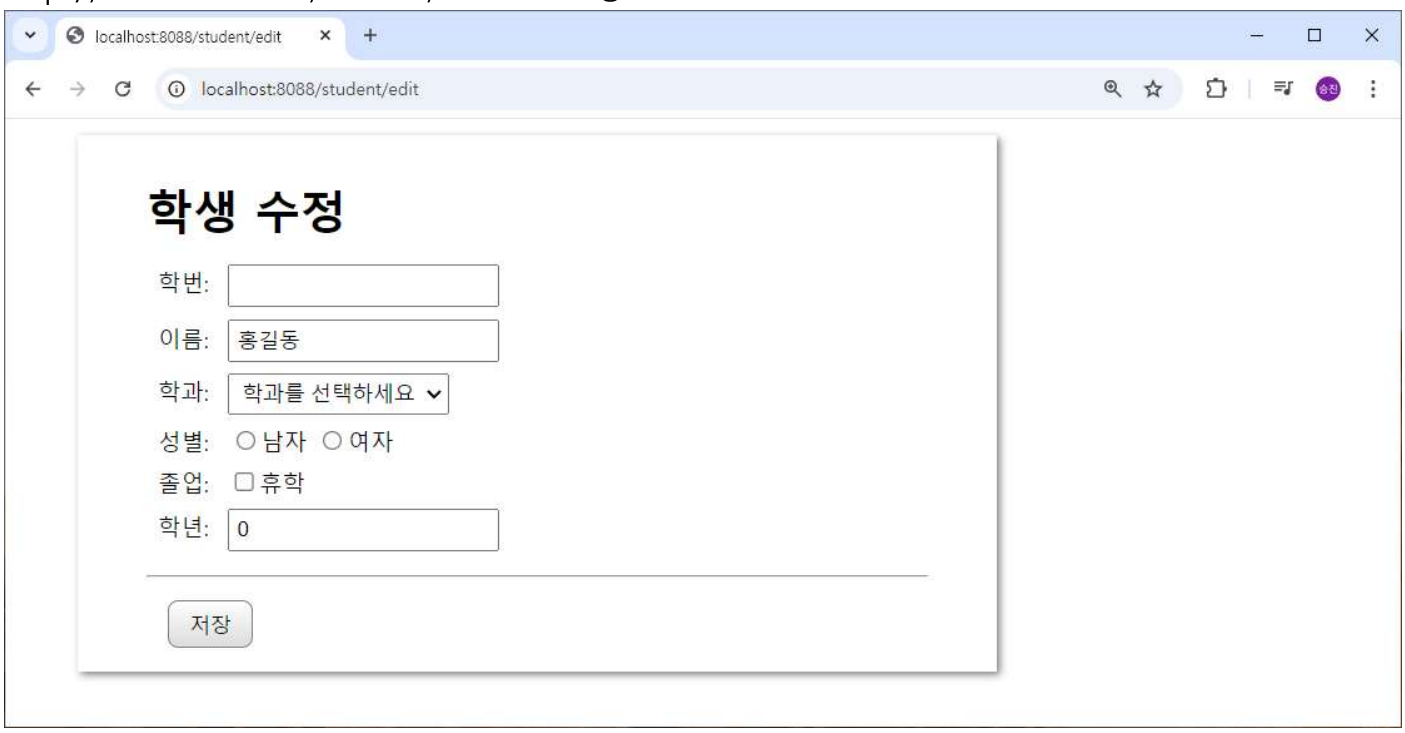
7) student/list.html

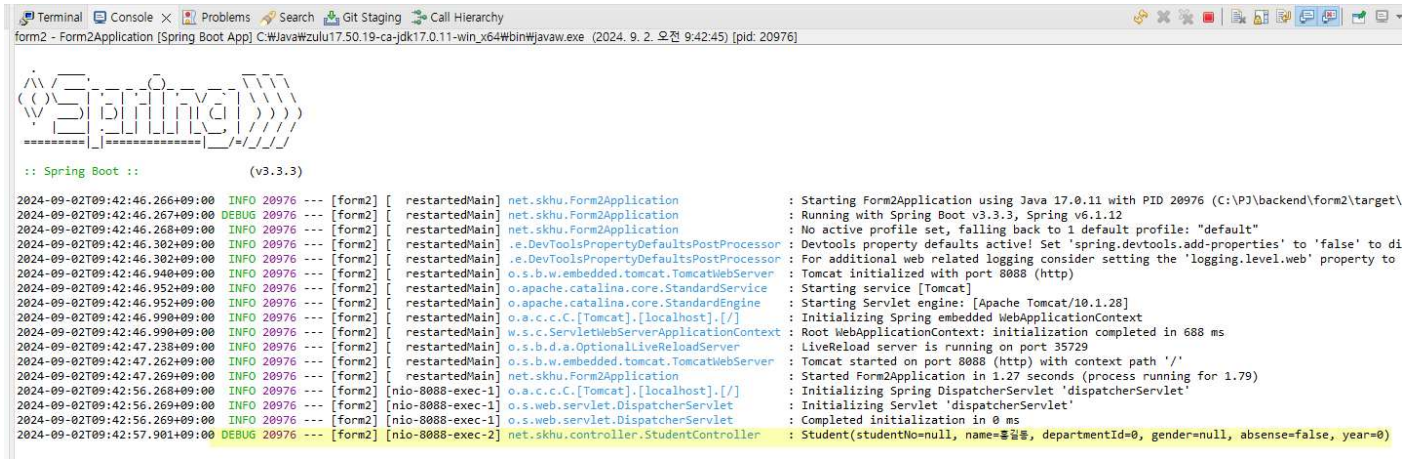
src/main/resources/templates/student/list.html

```
<!DOCTYPE html>
<html lang="ko" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8">
  <link rel="stylesheet" type="text/css" href="/common.css" />
  <style>
     div.container { width: 500px; padding: 2em; }
  </style>
</head>
<body>
<div class="container shadow">
   <h1>학생 목록</h1>
   <p>학생 목록 출력 생략</p>
</div>
</body>
</html>
```

8) 실행

http://localhost:8088/student/edit GET 요청





Console 창에 로그 메시지가 출력되었다.
  StudentController의 줄21 로그 출력

입력 폼에 데이터를 입력하고 저장 버튼을 클릭하여
  StudentController의 줄29 로그 출력을 확인해 보자.

application.properties 파일에서
  logging.level.net.skhu=debug 으로 설정했기 때문에
  debug 등급에 로그 메시지가 출력되었다.

  이 설정을 info 등급으로 변경하거나 주석처리하고 다시 실행해 보자.
  디폴트 등급이 info 등급이므로 debug 등급 로그가 출력되지 않을 것이다.

9) 연습문제

연습 #1

처음에 student/edit 입력 폼이 웹브라우저 창에 뜰 때,
홍길동 대신 자신의 이름이 자동으로 채워져 있도록 수정하라.

이름 뿐만 아니라, 자신의 학번, 학과, 성별, 학년 등도 자동으로 채워져 있도록 수정하라.

연습 #2

학번, 이름 뿐만 아니라
학과, 성별을 입력하지 않았을 때도 에러 메시지가 출력되도록 수정하라.

3. 부록

1) lombok

DTO 클래스에 @Data 어노테이션을 붙이면
  모든 멤버 변수에 대해서
  getter, setter, equals, hashCode, toString 메소드들이 자동 구현된다.

만약 자동으로 구현되는 메소드에서 제외되어야 할 멤버 변수가 있다면 다음과 같이 구현한다.
예:

```
@Data
@EqualsAndHashCode(exclude="temp")
@ToString(exclude="temp")
public class Student {
   int id;
   String name;
   int temp;
}
```

equals, hashCode, toString 메소드가 자동으로 구현될 때, temp 멤버 변수는 무시된다.

예:

```
@Data
@EqualsAndHashCode(exclude={"temp1","temp2"})
@ToString(exclude={"temp1","temp2"})
public class Student {
   int id;
   String name;
   int temp1;
   String temp2;
}
```

equals, hashCode, toString 메소드가 자동으로 구현될 때, temp1, temp2 멤버 변수는 무시된다.

2) 로그 메시지 출력

format 문자열

```
log.debug("student DTO 값 {}", student);
    student.toString() 값이 {} 부분에 출력된다.

log.debug("student DTO 값1 {}, 값2 {}", student1, student2);
    student1.toString() 값이 첫째 {} 부분에
    student2.toString() 값이 둘째 {} 부분에 출력된다.
```

에러 메시지 출력

```
log.error("에러 메시지", exception);
     에러 메시지뿐만 아니라
     exception 객체에 들어있는 자세한 에러 정보도 출력된다.


log.error("등록 실패 {}", student, exception);
     student.toString() 값이 {} 부분에 출력된다.
     exception 객체에 들어있는 자세한 에러 정보도 출력된다.
```